Title: Bridging Databases and Spreadsheets: Query Translation Techniques

Name of the Faculty: **Mr. Suraj Bhoite**

Abstract:

One of the most widely used programs for managing and analyzing data is Excel. They combine data processing with very diverse supplementary features: statistics, visualization, reporting, linear programming solvers, Web queries periodically downloading data from external sources, etc. However, the spreadsheet paradigm of computation still lacks sufficient analysis. In this work, demonstrate that a spread- sheet can implement all data transformations definable in SQL, merely by utilizing spread- sheet formulas. Offer a query compiler that can convert any SQL query into a worksheet with the same semantics including NULL values for any given query. Users that choose not to migrate to a database can now access database operations. They can define their queries using a high-level language and then get their execution plans in a plain vanilla spreadsheet. The functions available in spreadsheets impose limitations on the algorithms one can implement.

COPYRIGHT OFFICE
NEW DELHI
Reg. No. - LD-20250174511
Date 26/09/2025

Keywords: *Relational Databases, Physical database design prototypes, Spreadsheets, Query languages*

Objective:

The main aim of this System is translating relational queries into spreadsheets and can define their queries using a high level language and then get their execution plan in spread- sheet. Spreadsheets are the desktop equivalent of OLAP and databases in large-scale enterprise computing. They serve basically the same purpose data management and analysis but at the opposite extreme of the data quality scale. Demonstrate that a spreadsheet can implement all data transformation definable in SQL merely by utilizing spreadsheets formulas.

Motivation

The main group of potential users of this work to be characterized by the following:

- They are experienced and relatively proficient users of spreadsheets, mainly Microsoft Excel.
- They are approaching the limits of spreadsheet abilities.
- Either they are not yet ready to migrate to a database, or they do not want to migrate at all.

Applications:

1. Educational purpose
2. Banking purpose
3. [illegible]pilation over lightweight data
4. [illegible]structured information from spreadsheets

Architecture Diagram:

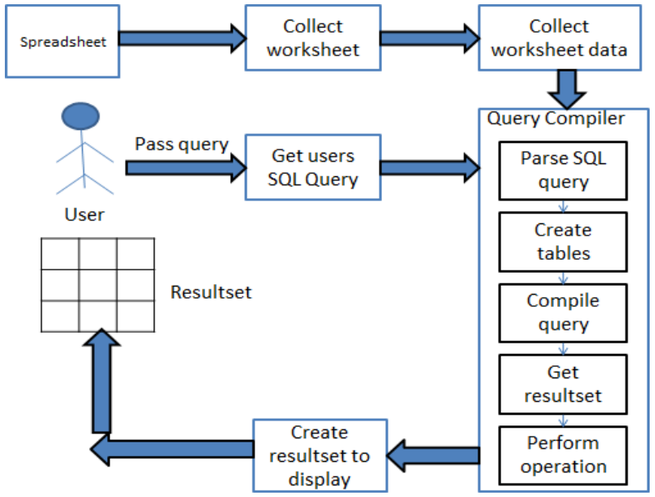

Figure: System Architecture

This System is number of minor issues arising in a practical implementation of the database operations in a spreadsheet. There is the limitation on the number and sizes of relations, views and their intermediate results, imposed by the maximal available number of worksheets, columns and rows in the spreadsheet. Next, the size of the data values is also limited. The variety of data types in spreadsheets is also restricted when compared to database systems.

The architecture of a relational database implemented in a spreadsheet is a specification of a query in the relational algebra, its implementation is created by query compiler, in the form of an .xlsx file. The resulting spreadsheet implementation of the query is a single worksheet, consisting of the necessary number of columns for the data tables and, the columns performing the computations. Besides the result columns of the query, the view worksheet typically contains a number of columns, which store intermediate results emerging during query evaluation.

Step1: Create Spreadsheet which contains number of worksheet.
Step2: Collect Worksheet and their data.
Step3: Construct a query compiler that applies several operations to a user-provided query.
Step4: User pass their SQL Query to the compiler.
Step5: Compiler take user query and perform following various operations:
1. Parse SQL query.
2. Create tables.
3. Compile query.
4. Get Result set.
5. Perform operations: Aggregation, Composition, join etc.

Step6: After compiling query create final Result set to display to user.

Mathematical Model:

Let Y be the system, we mathematically represent Y.

S={A, R, P, C, Cs}

Where,

S = Worksheets, A = Attributes in query string,
R = Relations in query string, P = predicates in query string,
C = conditions in query string, Cs = constants in query string.

Input Sets:

S= $\{w_0, w_1, \ldots w_{n-1}; 0 \le i \le n\}$
Where,
n = number of worksheets.

A = $\{a_0, a_1, \ldots a_{n-1}; 0 \le i \le n\}$
Where,
n= number of attributes in query string.

R = $\{r_0, r_1, \ldots r_{n-1}; 0 \le i \le n\}$
Where,
n= number of relations in query string.

P = $\{p_0, p_1, \ldots p_{n-1s}; 0 \le i \le n\}$
Where,
n= number of predicates in query string.

C = $\{c_0, c_1, \ldots c_{n-1}; 0 \le in\}$
Where,
n= number of conditions in query string.

Cs = $\{cs_0, cs_1 \ldots cs_{n-1}; 0 \le i \le n\}$
Where,
n= number of constants in query string.

Output sets:

R = $\{v_{ij} | 0 \le i \le n; 0 \le j \le m\}$
Where,
n=no. of attribute m=no. of rows

Let Y be the system, we mathematically represent Y in set theory as,
Y = $\{S, A, R, P, C, Cs; W, R\}$

Conclusion:

Established that SQL can be automatically converted into spreadsheet code. Thus, they have demonstrated the strength of the spreadsheet paradigm, which supplants the relational database paradigm. Apart from SQL, the bubble sort method has also been used by them. The next steps should involve creating spreadsheet translations for SQL queries and optimizing them. They are also interested whether spreadsheets can naturally implement other models of databases, like semi-structured or object-relational ones. This requires translating Google specific functions query, sort, filter and unique which are not recognized by other spreadsheet systems